Hierarchical DBMS

Data models in database systems

Lecture 8

Mara Romanovska
Mara.Romanovska@rtu.lv

27.11.2025.

1862
RTU
DATORZINĀTNES UN
INFORMĀCIJAS
TEHNOLOĢIJAS FAKULTĀTE

Content

- XQuery
- Non-native XML databases and hybrids
- Native document databases

Querying XML data: XQuery

XQuery in general

- XQuery is a **query and transformation language** designed specifically for working with XML data
- It allows you to search, filter, extract, and restructure XML
- Works on XML (and XML-like) data sources and is itself **SQL-like**
- Uses **XPath** expressions for navigation
- Supports **FLWOR expressions** (for–let–where–order by–return) similar to SQL-style querying
- Can generate new XML, HTML, JSON, or text as output

Let Clause

let $var := expr

Example

```
let $ bk := doc("book.xml"),
    $ bka := doc("book.xml")/catalog//author
```

For Clause

for $var in expr

Example

```
let $bk :=doc("book.xml")
for $b in $bk/catalog//author
    ...
```

for $x in doc("books.xml")/biblio/author[name = "Jeff"]/book/title (:all the books by Jeff:)
for $y in doc("books.xml")/biblio/author/book/title (:all the books:)

Where Clause

where expr

- Similar to WHERE clause in SQL
- Comparisons
  - General comparisons: =, !=, <, <=, >, >=
  - Value comparisons: eq, ne, lt, le, gt, ge

```
for $x in doc("books.xml")/biblio/author[name = "Jeff"]/book/title (:all the books by Jeff:)
for $y in doc("books.xml")/biblio/author/book/title                (:all the books:)
where data($x) = data($y)
return $y
```

Return Clause

return expr

- Evaluated once for every item in the sequence
- The final result is an ordered sequence containing the results of these evaluations
- The RETURN clause is typically executed multiple times inside FOR loops

Example

```
let $bk :=doc("book.xml")
for $b in $bk/catalog//book
where $b/author="John"
return $b/title
```

Example XML – Movies and Stars

```
1)  <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2)  <Movies>
3)      <Movie title = "King Kong">
4)          <Version year = "1933">
5)              <Star>Fay Wray</Star>
6)          </Version>
7)          <Version year = "1976">
8)              <Star>Jeff Bridges</Star>
9)              <Star>Jessica Lange</Star>
10)         </Version>
11)         <Version year = "2005" />
12)     </Movie>
13)     <Movie title = "Footloose">
14)         <Version year = "1984">
15)             <Star>Kevin Bacon</Star>
16)             <Star>John Lithgow</Star>
17)             <Star>Sarah Jessica Parker</Star>
18)         </Version>
19)     </Movie>
20) </Movies>
```

```
1)  <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2)  <Stars>
3)      <Star>
4)          <Name>Carrie Fisher</Name>
5)          <Address>
6)              <Street>123 Maple St.</Street>
7)              <City>Hollywood</City>
8)          </Address>
9)          <Address>
10)             <Street>5 Locust Ln.</Street>
11)             <City>Malibu</City>
12)         </Address>
13)     </Star>
            ... more stars
14) </Stars>
```

Joins in XQuery

Example

```
let $m := doc("movies.xml"),
    $s := doc("stars.xml")
for $s1 in $m/Movie//Star,
    $s2 in $s/Star
where data($s1)=data($s2/Name)
return $s2/Address/City
```

Eliminate Duplicates

- Built-in function **distinct-values**
- To include another expression in the result, use curly braces

Example

```
let $stars := distinct-values(
    let $movies := doc("movies.xml")
    for $m in $movies/Movies/Movie
    return $m/Version/Star)
return <Stars>{$stars}</Stars>
```

Aggregation

Example

```
let $movies := doc("movies.xml")
for $m in $movies/Movies/Movie
where count($m/Version) >2
return $m
```

Group By

group by expr

```
let $x := 64000
for $c in //customer
let $d := $c/department
where $c/salary > $x
group by $d
return
  <department name="{$d}">
     Number of employees earning more than ${distinct-values($x)} is {count($c)}
  </department>
```

Branching

if (expr1) then expr2 else expr3

- Every expression must return a sequence, else is mandatory
- If there is nothing to return, return an empty sequence ()

Example

```
let $m :=
doc("movies.xml")/Movies/Movie[@title="King Kong"]
for $v in $m/Version
return
    if ($v/@year=max($m/Version/@year))
    then <latest>{$v}</latest>
    else <old>{$v}</old>
```

Use XPath in predicate []

Example

```
let $m :=
doc("movies.xml")/Movies/Movie
for $v in $m/Version
return $v/year[.=2010]
```

Example

```
xquery version "3.1";

(: Find the names of all Jeff’s co-authors and list them together with the titles of books that were coauthored.  :)

for $lol in distinct-values
(for $x in doc("books.xml")/biblio/author[name = "Jeff"]/book/title (:all the books by Jeff:)
for $y in doc("books.xml")/biblio/author/book/title                (:all the books:)
where data($x) = data($y)
return $y)
for $i in doc("books.xml")/biblio/author
where data($lol) = data($i/book/title)
group by $lol
return if (count($i)>1)
        then
        (<book>
        <title> {$lol} </title>
        {$i/name}
        </book>)
        else()
```

XML databases

There are two ways for storing DoD

- Document-enabled databases documents are transformed into universal DB storage structures (such as relational)
- Native documents databases where internal models are specifically tailored for the document storage

Working with documents in Oracle

Storage options

- **Structured** storage – XMLType data is stored as a set of objects
- **Unstructured** storage – XMLType data is stored in Character Large Object (CLOB) Instances
- **Binary XML** storage – XMLType data is stored in a post-parse, binary format specifically designed for XML data

Pros and cons: unstructured storage

- Pros:
  - enables higher throughput than structured storage when inserting and retrieving entire XML documents.
  - provides greater flexibility than structured storage in the structure of the XML

- Cons:
  - No query optimization
  - No updates on XML data

Unstructured storage is particularly appropriate for document-centric use cases.

Pros and cons: structured storage

- Pros:
  - near-relational query and update performance,
  - optimized memory management,
  - reduced storage requirements,
  - B-tree indexing,
  - and in-place updates.
- Cons:
  - increased processing overhead during insertion and full retrieval of XML data,
  - reduced flexibility in the structure

- **Appropriate for highly structured data whose structure does not vary**

Pros and cons: binary XML storage

- Binary XML storage
  - provides more efficient database storage, updating, indexing, and fragment extraction than unstructured storage
  - «understands» schemas
  - allows piece-wise updates
  - allows for very complex and variable data

- **You can use binary storage for XML schema-based data even if the XML schema is not known beforehand**

XML Type

- **XMLType** is an abstract native SQL data type for XML data.
- XMLType can be used as any other type:
  - To create a column in a relational table
  - To declare a PL/SQL variable
  - To define or call a PL/SQL procedure or function
- XMLType provide:
  - Constructors that allow transforming other types to XML
  - XML-specific methods that operate on XMLType instances

XML file+Schema

XML file

```xml
<?xml version="1.0" encoding="UTF-8"?>

<!-- New XML document created with EditiX XML Editor (http://www.editix.com) at Mon Dec 02 18:08:59 EET 2019 -->
<videostore>
    <movie category="Comedy">
            <title lang="eng">Home Alone</title>
            <director>Chris Columbus</director>
            <year>1990</year>
            <price>15.00</price>
    </movie>
    <movie category="Action">
            <title lang="eng">Die Hard</title>
            <director>John McTiernan</director>
            <year>1988</year>
            <price>10.00</price>
    </movie>
</videostore>
```

XML Schema

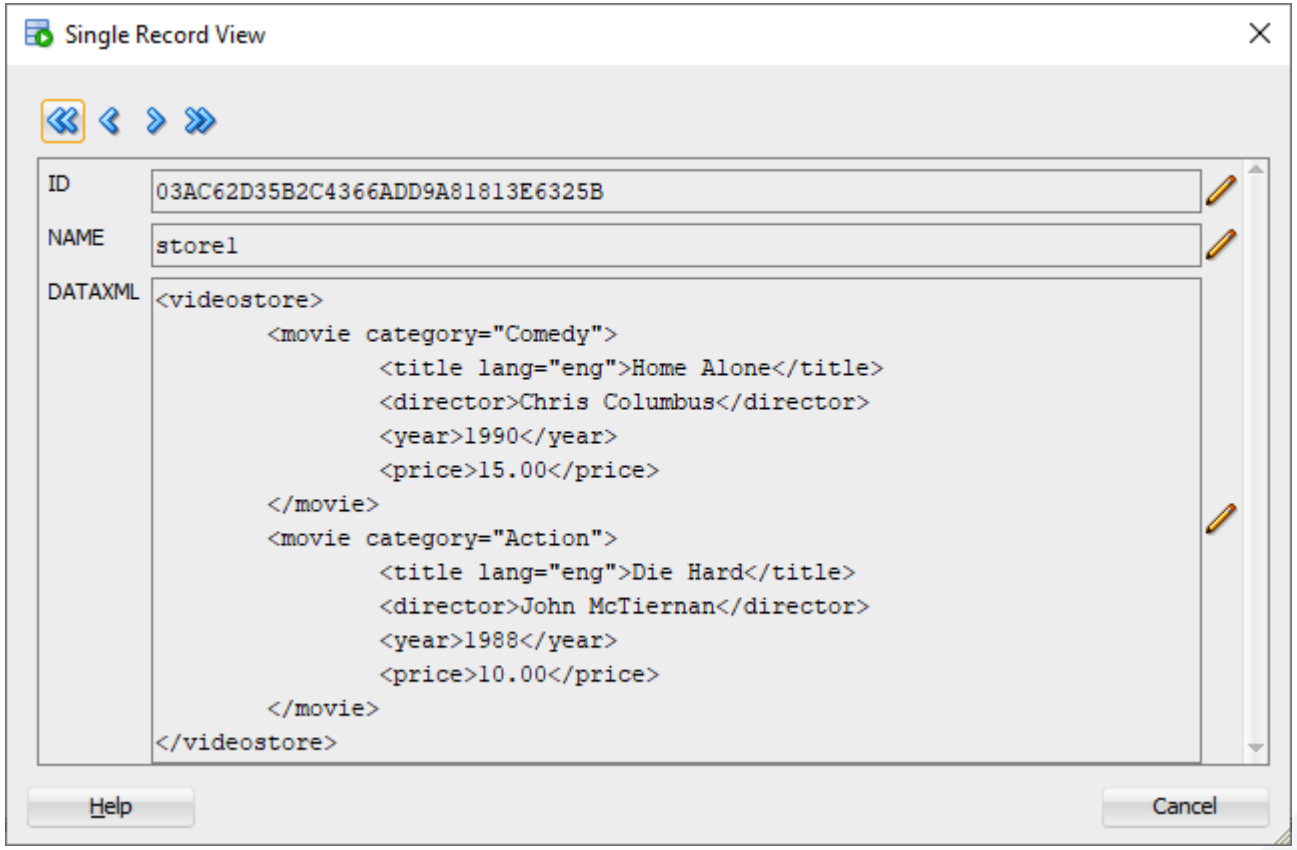
```xml
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema elementFormDefault="qualified" xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="movie">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="title"/>
                <xs:element ref="director"/>
                <xs:element ref="year"/>
                <xs:element ref="price"/>
            </xs:sequence>
            <xs:attribute name="category" type="xs:string" use="required"/>
        </xs:complexType>
    </xs:element>
    <xs:element name="year" type="xs:string"/>
    <xs:element name="director" type="xs:string"/>
    <xs:element name="price" type="xs:string"/>
    <xs:element name="title">
        <xs:complexType>
            <xs:simpleContent>
                <xs:extension base="xs:string">
                    <xs:attribute name="lang" type="xs:string" use="required"/>
                </xs:extension>
            </xs:simpleContent>
        </xs:complexType>
    </xs:element>
    <xs:element name="videostore">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="movie" maxOccurs="unbounded"/>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

Unstructured storage in practice

```
create table VIDEO_STORES_CLOB(
ID    varchar2(40),
NAME    varchar2(20),
DATAXML XMLType)
XMLTYPE COLUMN DATAXML store as CLOB;
```

```
insert into VIDEO_STORES_CLOB(ID, NAME,
DATAXML) values (sys_guid(), 'store1',
'<videostore>
      <movie category="Comedy">
            <title lang="eng">Home Alone</title>
            <director>Chris Columbus</director>
            <year>1990</year>
            <price>15.00</price>
      </movie>
      <movie category="Action">
            <title lang="eng">Die Hard</title>
            <director>John McTiernan</director>
            <year>1988</year>
            <price>10.00</price>
      </movie>
</videostore>');
```

Result (SELECT *...)



Structured storage in practice

- Schema registration

```
<xs:schema elementFormDefault="qualified" xmlns:xs="http://www.w3.org/2001/XMLSchema">
      <xs:element name="movie">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="title"/>
                <xs:element ref="director"/>
                <xs:element ref="year"/>
                <xs:element ref="price"/>
            </xs:sequence>
            <xs:attribute name="category" type="xs:string" use="required"/>
        </xs:complexType>
    </xs:element>
    <xs:element name="year" type="xs:string"/>
    <xs:element name="director" type="xs:string"/>
    <xs:element name="price" type="xs:string"/>
    <xs:element name="title">
        <xs:complexType>
            <xs:simpleContent>
                <xs:extension base="xs:string">
                    <xs:attribute name="lang" type="xs:string" use="required"/>
                </xs:extension>
            </xs:simpleContent>
        </xs:complexType>
    </xs:element>
    <xs:element name="videostore">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="movie" maxOccurs="unbounded"/>
            </xs:sequence>
         </xs:complexType>
    </xs:element>
</xs:schema>
```

```
Worksheet  Query Builder
BEGIN
  DBMS_XMLSCHEMA.registerSchema(
    SCHEMAURL => 'XML_SCHEMA_1',
    SCHEMADOC => '<xs:schema elementFormDefault="qualified" xmlns:xs="http://www.w3.org/2001/XMLSchema">
      <xs:element name="movie">
        <xs:complexType>
            <xs:sequence>
                <xs:element ref="title"/>
                <xs:element ref="director"/>
                <xs:element ref="year"/>
                <xs:element ref="price"/>
            </xs:sequence>
            <xs:attribute name="category" type="xs:string" use="required"/>
        </xs:complexType>
    </xs:element>
    <xs:element name="year" type="xs:string"/>
    <xs:element name="director" type="xs:string"/>
    <xs:element name="price" type="xs:string"/>
...
    LOCAL => TRUE,
    GENTYPES => TRUE,
    GENTABLES => FALSE,
    CSID => nls_charset_id('AL32UTF8'));
END;
/
```

Registered schema and table

- Metadata retrieval

Worksheet | Query Builder

```
1  select ELEMENT_NAME, TABLE_NAME, STORAGE_TYPE
2  from USER_XML_TABLES
3  where XMLSCHEMA = 'XML_SCHEMA1';
```

Script Output | Query Result

SQL | All Rows Fetched: 7 in 2.745 seconds

	ELEMENT_NAME	TABLE_NAME	STORAGE_TYPE
1	videostore	VIDEOSTORES_SCHEMA	OBJECT-RELATIONAL
2	director	director805_TAB	CLOB
3	movie	movie808_TAB	OBJECT-RELATIONAL
4	price	price807_TAB	CLOB
5	title	title804_TAB	OBJECT-RELATIONAL
6	videostore	videostore810_TAB	OBJECT-RELATIONAL
7	year	year806_TAB	CLOB

```
create table VIDEOSTORES_SCHEMA of
XMLType
XMLSCHEMA "XML_SCHEMA1"
ELEMENT "videostore";

…STORE AS TABLE
```

Creating binary storage

- You can:
  - allow any schema (ANYSCHEMA)
  - store with no schema (NONSCHEMA)
  - combinations

```
create table VIDEO_STORES_BINARY of XMLType
    XMLTYPE store AS BINARY XML;

create table VIDEO_STORES_BINARY of XMLType
    XMLTYPE store AS BINARY XML
ALLOW ANYSCHEMA;
```

Querying Oracle XML DB

General options for querying XML in Oracle

- XMLType methods to get XML file to other formats
  - getStringVal();
  - getNumberVal();
  - getBLOBVal(csid);
- SQL functions
  - extract()
  - existsNode()
  - extractValue()
- SQL/XML standard functions such as XMLQuery, XMLTable, XMLExists, and XMLCast.
- Using PL/SQL or JAVA
- Using xQuery
- Text () operator

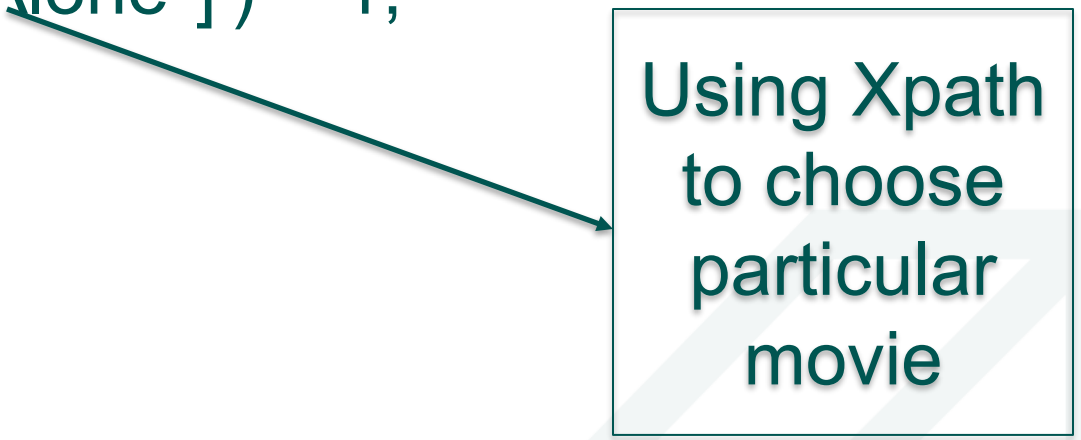
XMLType methods

```
select x.OBJECT_VALUE.getCLOBVal(‘/videostore')
from VIDEOSTORES_CLOB x;
```

Gets the value of the /videostore parameter in the CLOB.

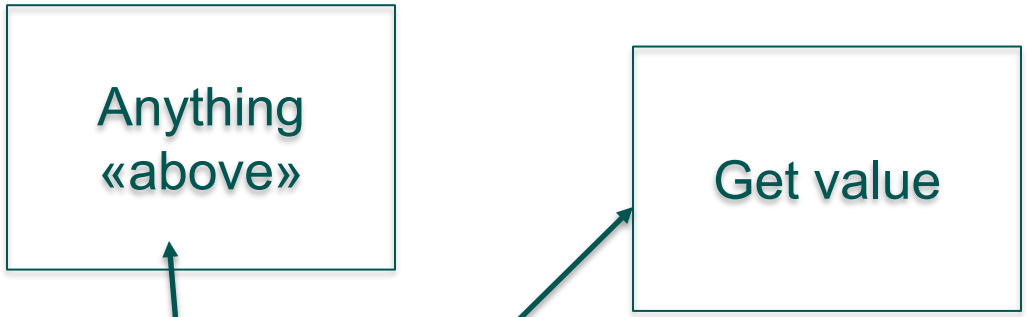
ExistsNode()

```
select OBJECT_VALUE
from VIDEO_STORES_TAB
where existsNode(OBJECT_VALUE,
'/videostore/movie[title="Home Alone"]') = 1;
```



Function extract()

```
select extract(OBJECT_VALUE, '/videostore/movie/title')
"TITLES"
from VIDEO_STORES_TAB;
```

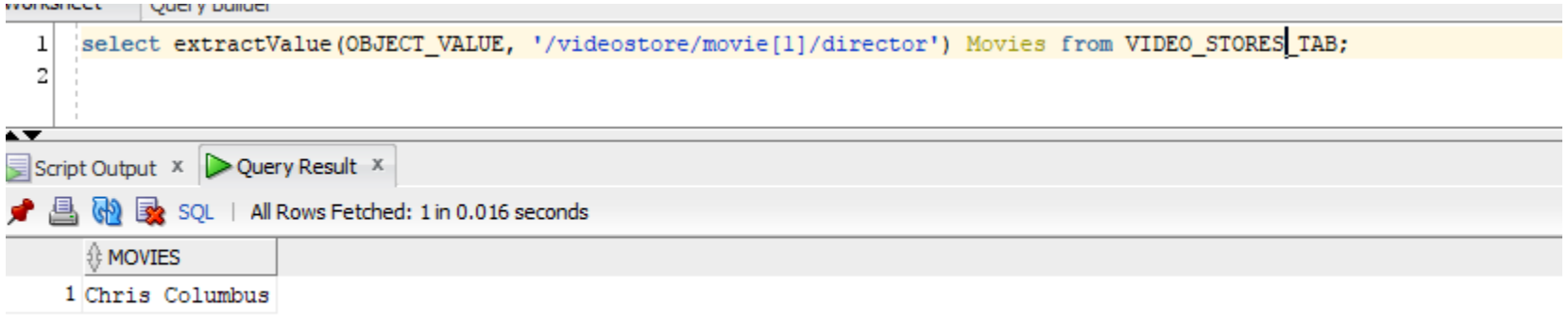



```
select extract(OBJECT_VALUE, '//movie/title/text()')
"TITLES"
from VIDEO_STORES_TAB
where existsNode(OBJECT_VALUE, '//movie[title="Home
Alone"]') = 1;
```

extractValue()

Just the first movie

```
select extractValue(OBJECT_VALUE,
'/videostore/movie[1]/director') Movies from
VIDEO_STORES_TAB;
```



Using SQL

```
select extractValue(OBJECT_VALUE,
'/videostore/movie[2]/director') DIR
from VIDEO_STORES_TAB
order by
extractValue(OBJECT_VALUE,'/videostore/movie[2]/director');
```

Relational/XML interchangeability

XML/SQL Duality

- XML/SQL duality means that the same data can be exposed as rows in a table and manipulated using SQL or exposed as nodes in an XML document

Relations to XML

```
select  xmlElement("videostore",
xmlAttributes(a.STORE as "name"),
xmlForest(a.MOVIE as "movie")
).extract('/*') as XML
from VIDEO_STORES_RELATIONAL  a;

XML
--------------------------------------
<videostore name="ABCMOVIES">
  <movie>DBS</movie>
</videostore>
```

Creating object table

```
create table VIDEO_STORES_TAB of XMLType;
```

```
insert into VIDEO_STORES_TAB values (
XMLType('
<videostore>
    <movie category="Comedy">
        <title lang="eng">Home Alone</title>
        <director>Chris Columbus</director>
        <year>1990</year>
        <price>15.00</price>
    </movie>
    <movie category="Action">
        <title lang="eng">Die Hard</title>
        <director>John McTiernan</director>
        <year>1988</year>
        <price>10.00</price>
    </movie>
</videostore>'));
```

```
select Value(a) from VIDEO_STORES_TAB a;
```

Native database storage

Can Oracle XML features be considered native?

- Yes! But not all XML documents stored in Oracle are stored in a native way

- **Native XML (application/database)** means that an application or system is specifically designed to work with XML data, treating an XML document as a **fundamental unit of storage and operation**

Main characteristics of native XML databases

- No standard, yet there are some common characteristics
  - Has an XML document as its fundamental unit of (logical) storage
  - Specific, list-like indexing
  - Standard implementation (supports schemas, XPath, XQuery)

Native XML DBMSs

- BaseX
- eXist

- Larger DBMSs, such as Oracle DBMS and IBMs DB2 offer native XML storage options

Thank you!